Predicting Changes in Waste Shipment Activity Using Machine Learning with Graphs

Owen Wurst
University of Texas
Email: owurst@utexas.edu
Code: https://github.com/OWurst/Case_Studies_Final_Proj

***Abstract*—This paper presents a machine learning approach for predicting changes in both nontoxic and toxic waste shipment to various treatment facilities using EPA RCRA historical logistic data. The goal is to predict changes in waste shipment for 25 waste types to 100 facilities using 50 generators and 500 transporters and compare prediction results using both traditional relational database systems and graph networks for various model types. Using data sourced originally from the EPA RCRA database that has been warehoused in PostgreSQL, we construct a pipeline to both train natively as well as build a graph network and train on that. Results demonstrate that using a graph network, at least in the form and with the framework we did, did not improve predictive results on our models.**

I. Introduction

The United States Environmental Protection Agency (EPA) has a centralized database known as RCRAInfo that is a source for tracking toxic and nontoxic waste for various generators, transporters, and treatment facilities [1]. This data is public and is meant to improve public safety and environmental regulation. Predicting changes in waste shipment activity allows the public to ensure safety and monitor waste prevelance in their communities as well as allowing private companies greater ability to identify potential opportunities to acquire waste from one company that may actually be a necessary component in their own operation. Overall, these predictions lead to an increase in reuse of materials, especially with regards to nontoxic waste, and a safer society, especially with regards to toxic waste.

In thinking about how a very intelligent human would predict changes in waste shipping to certain treatment and management facilities, I began to think about how they would look outside the facility itself and the changes in waste amounts and instead look at the producers and transporters that are likely to ship to these facilities. For example, if a nearby factory is expanding and increasing production of product A, of which a side effect of production is converting chemical X to chemical Y, then we are likely to see an increased demand of chemical X and an increase in the output of chemical Y, which in this case we will assume is some toxic waste. With this knowledge, we would seek to find which transporters the factory most commonly uses to transport chemical Y, because they are likely to see an influx as well. Finally, we would want to know which treatment centers these transporters most commonly send chemical Y to, because they too are likely to see an influx. Through this and similar methods, the very intelligent human would begin to predict change in shipments of chemical Y, and any other chemicals, to specific treatment facilities, the idea would be the same for shrinking and closing factories, and so on. These are the kind of inter-facility relationships we are hoping to embed within our graph network.

Graph networks are networks that seek to store data in terms of the relationships between nodes rather than simple tables that must be joined in an attempt to extrapolate those relationships [2]. In organizing the data into a graph, I expect to see better results because we will be able to enrich our data with information about the relationships between the facilities that produce waste, the transporters they use to ship waste, and the facilities that ultimately treat the waste. Machine learning models used will then, through a graph building and dataloading pipeline, receive additional information about producers and transporters implicitly through the network and essentially have the power to rapidly automate the thinking process of the very intelligent human described above.

A. Research Question

Can machine learning models more accurately predict changes in waste shipment activity using historical logistics data when data is organized as a graph rather than a traditional relational database?

B. Contributions

- A dataloading method for converting RCRA EPA data into a graph network
- A predictive framework for modeling changes in shipment activity for both graph databases and
- A feature engineering and data loading pipeline for RCRA EPA data with python
- An empirical evaluation for various ML methods on both graph and classical data structures

II. Related Work

While there has not been work done specifically with graph machine learning on this EPA data there has been a variety of work predicting changes in shipments and waste storage volumes with the RCRA data as a source in isolated contexts (often focusing on a specific region or chemical). Additionally, there has been a large body of work in using graphs and graph based machine learning to model various supply chain and

shipping problems that are similar to this problem, and we will examine these two bodies of research separately.

A. EPA ML

The first and smaller body of research has been on general ML and data mining applications with this EPA data.

The first paper I found demonstrated through a study of waste disposal site failures the necessity of tracking hazardous substances with ML in part because of the degree of difficulty and complexity in tracking these substances manually [3]. This exemplifies the need for projects like the one that I have undertaken.

The second paper I found was in part on predicting quantities and specific locations (treatment centers) of mercury in the Portland area. As mercury has the potential to be a toxic chemical, the goal of the paper was to provide a method to predict and track mercury volumes by following another type of waste, cement kilns, from the sources to the transporters to the final facilities; however, the primary finding in this paper is that this is a highly uncertain prediction task and that their models were unable to achieve a high degree of accuracy [4].

The third paper, while not focused on ML specifically, focused on data mining of this EPA data and creating categories of features [5] within the RCRA database apart from their schema to identify feature importance. From this paper, I identified several features like "management method" that will hopefully be useful in my classification task.

B. Graph ML

Within Graph Machine learning, there seems to be two disciplines that problems tend to be somewhere on the spectrum between. The first is with problems that align more towards predicting the edges, or predicting a path through the nodes, and the second aligns more towards predicting what will happen at a given node based on the broader network. The reason both are critical is that most graph problems, including mine, will require approximating and aggregating information about both nodes and edges related to your target object regardless of whether it is a node or edge.

In the first type of paper, I found one that showed that using graph based machine learning in supply chain management was able to directionally predict an outcome using the surrounding edges of a node was more successful than when using ML on features of just the node individually [6]. Another paper showed that when partitioning and aggregating nodes based carefully selected target features that both training and prediction could be completed much faster with minimal loss [7]. Additionally, in a paper predicting medicare fraud, it was shown that when comparing targeted graph machine learning with properly selected features to machine learning methods that contained all those features in a tabular structure, the learning algorithms face less scalability concerns and can thus support more features implicitly [8]. The medicare fraud paper had the same findings as another a paper on intelligent manufacturing that was focused on solving the same scalability problems [9] as well as a paper focused on supply chain management in disaster scenarios [10]. Conversely, two papers, one on electronic data retrieval [11] and one on cognitive supply chain innovations [12] argued that rather than rather than careful graph construction and proper aggregation, focus needs to be on data enrichment to carefully protect against edge cases having too great an effect on training. From this set of papers, it is clear that I must both construct the graph I build carefully with the correct node and edge architecture but also focus on specific cases or else my models will struggle to train adequately.

In the second type of paper which is more geared towards prediction of what happens at an individual node at some time step, there are examples in training more geared towards what I must do not just in graph construction but at training time and beyond, in implementation. One paper showed that specifically neural networks are far better than other models on graphs because they are less likely to examine features individually but instead as relationary [13]. This paper was by a freight transportation network study that found that specifically convolutional neural networks are idea for graph modeling [14]. A paper on power distribution also found the same result, with graph neural networks specifically being the best [15].

Three papers focused specifically on graph-native ML algorithms. One of them was on retail sales demand [16] and one was a research paper on supply chain optimization [17], both of which showed better predictive performance when using graph native algorithms rather than traditional ML methods. Conversely, one research paper on graph topologies themselves showed that non graph native methodologies performed comparably to the graph-native ones but that there was more risk of overfitting [18].

III. Materials and Data Sources

A. Data Sourcing

The dataset consists of waste shipment and handling records stored in PostgreSQL, including entities such as facilities, handlers, and manifests. The data was sourced from a private company's warehouse of the EPA RCRA data that was extracted into postgres with python. This was done in order to limit the load on the company's servers and provide a sandbox in which I could operate more cleanly without fear of error. Within the sandbox, I was able to build the graph and views necessary for training.

B. Tools

Postgres was chosen as a database management system because it works well with python, my most functional language [19] and because it has strong support to mock graph databases without requiring a database management system specifically built for graphs [20].

C. Control Data Overview

Fields:

- uid

- year_quarter
- quarter_start
- facility_epaid
- waste_code
- waste_line_row_count
- manifest_count
- generator_count
- qty_tons
- avg_line_qty_tons
- std_line_qty_tons
- mode_management_method_code
- mode_form_code
- mode_source_code
- qty_lag_1
- qty_lag_2
- qty_lag_3
- qty_lag_4
- lag_qty_sum
- qty_vs_lag_avg_ratio
- next_qty_tons
- target_class

TABLE I
CLASS DISTRIBUTION: CONTROL DATA

Total Records	Increasing	Decreasing	Static
35,586	15,405	16,660	3,521

D. Graph Data Overview

Fields:

- uid
- year_quarter
- year
- quarter
- facility_epaid
- facility_node_id
- waste_stream_id
- waste_stream_key
- display_name
- management_method_code
- form_code
- source_code
- manifest_count
- waste_line_count
- total_waste_tons
- total_waste_kg
- unique_transporters_from_gf
- first_shipped_date
- last_shipped_date
- generator_pair_count
- generator_count
- pair_manifest_count_sum
- pair_waste_line_count_sum
- max_generator_pair_tons
- avg_generator_pair_tons
- repeated_generator_pairs
- summed_unique_transporters_from_pairs
- transporter_pair_count
- transporter_count
- trans_fac_manifest_count_sum
- trans_fac_waste_line_count_sum
- max_transporter_pair_tons
- avg_transporter_pair_tons
- repeated_transporter_pairs
- summed_unique_generators_from_pairs
- global_gen_trans_pair_count_for_stream_qtr
- global_gen_trans_generator_count_for_stream_qtr
- global_gen_trans_transporter_count_for_stream_qtr
- global_gen_trans_manifest_count_sum_for_stream_qtr
- global_gen_trans_waste_line_count_sum_for_stream_qtr
- global_max_gen_trans_pair_tons_for_stream_qtr
- qty_lag_1
- qty_lag_2
- qty_lag_3
- qty_lag_4
- manifest_count_lag_1
- generator_count_lag_1
- transporter_count_lag_1
- generator_pair_count_lag_1
- transporter_pair_count_lag_1
- lag_qty_sum
- qty_vs_lag_avg_ratio
- manifest_count_qoq_delta
- generator_count_qoq_delta
- transporter_count_qoq_delta
- generator_pair_count_qoq_delta
- transporter_pair_count_qoq_delta
- next_qty_tons
- target_class

TABLE II
CLASS DISTRIBUTION: GRAPH DATA

Total Records	Increasing	Decreasing	Static
673,410	271,485	289,604	112,321

IV. METHODOLOGY

A. ETL

The first, most difficult, and most time consuming portion of the project was building the extract, transform, and load pipelines required to organize the data as well as build the graphs.

The first step of this was to get the data from the source company's sql server warehouse into my own local PostgreSQL database. This is a common ETL problem and has been well researched [21]. This process requires running a python script to first read the schema of the source database and build corresponding tables in the destination database. Finally, because the database is $\tilde{3}$00GB, the data must be batch loaded into the destination database.

Next, I created a materialized view for the control database which represents our more traditional relational database model. In this view we join information like facility, waste type, and get lag quantities (see III.C.) to give us a temporal view of how things might be changing over time. This table was built using the top 25 waste codes, top 100 treatment facilities, top 50 waste generators, and top 500 waste transporters.

Third, I had to convert the data into a graph. Converting a relational database to a graph database is not currently a deterministically solved problem [22], and what tools do exist are designed for true graph databases like Neo4j rather than abstractions like postgres [23]; however, it is a well-studied procedure with a roadmap that involves conceptualizing the data in terms of nodes and edges [24]. For my nodes, these are the transporters, generators, and treatment facilities. For the edges, each edge is a transport stream for each given generator-transporter-facility-waste_type tuple for a given quarter weighted with the volume of a given waste type for that tuple in the given quarter.

Finally, I created the materialized view for the graph database. In this view we aggregate edges to a given facility in that quarter (and the preceding 4 quarters), again for the top 25 waste codes, top 100 treatment facilities, top 50 waste generators, and top 500 waste transporters.

B. Training Pipelines

To train, tune, and test the the models, it was necessary to build a pipeline that would read from the views using an offset to efficiently limit and batch read the data. The data is split on each batch into train (70%) validation (15%) and test (15%) data to ensure that we are safely loading batches and limiting data leakage by guaranteeing that the dataloader will not pull a record in validation or testing that the model was trained on despite the fact that all records are within the same table.

Because of the size of the training data, models needed to be saved, garbage collected, and reloaded at various times in order to protect against memory errors when the training pipeline was training multiple models sequentially. This added time to training but allowed alterations to data, the evaluation process, and the model tuning to be done safely without overwriting existing functional models.

C. Models

Because our datasets are too large to fit in memory all at once, particularly in the case of the graph data set, I have chosen to evaluate prebuilt models that explicitly allow incremental learning [25], which will allow me to train models batch by batch using python [26]. For simpler baseline models, we have chosen to use the sklearn library because it is easy to use and has a wide variety of model choices, and for batch learning, the chosen models must implement the partial_fit() function [25]. For our more complex neural network, we have chosen to use the keras library because it provides simple tools for training neural networks. When using keras with batch learning, we need to select a model that implements the train_on_batch method [27].

To this end, we have chosen to evaluate the following 3 models:

- Sklearn SGDClassifier: a model that uses Stochastic Gradient Descent to train a linear model as a classifier [28].
- Sklearn Multinomial Naive Bayes: the Naive Bayes model is used to classify data with discrete features [29].
- Keras Sequential Model: the keras sequential model is used to build a classical neural network with dense layers [30].

As our baseline models, we have chosen to try using both the SGDClassifier and Naive Bayes Classifier because while our training samples are split into discrete categories (a specific waste type seeing an increasing, decreasing, or static shipment volume to a given facility) outputs are on a sliding scale, so it will be interesting to see if the linear classifier achieves better results by seeing this sliding scale or if the bayesian classifier achieves better results by viewing these classes as discrete.

D. Evaluation Metrics

For evaluation, since I have chosen to classify training samples into one of 3 classes, I will use the following standard classification metrics:

- Accuracy
- Precision
- Recall
- F1-score

Additionally, we will see a confusion matrix for each of the models in order to better visualize how the models are performing on specific classes.

V. Results

TABLE III
MODEL PERFORMANCE: CONTROL

Model	Accuracy	Precision	Recall	F1
SGD	0.51	0.67 0.16 0.47	0.32 0.033 0.85	0.44 0.056 0.60
Multinomial NB	0.50	0.56 0.22 0.55	0.60 0.32 0.43	0.58 0.26 0.48
Sequential NN	0.52	0.52 0.29 0.52	0.74 0.08 0.37	0.61 0.13 0.43

TABLE IV
MODEL PERFORMANCE: GRAPH

Model	Accuracy	Precision	Recall	F1
SGD	0.48	0.48 0 0	1 0 0	0.65 0 0
Multinomial NB	0.45	0.46 0.05 0.45	0.01 0.02 0.97	0.02 0.03 0.62
Sequential NN	0.45	0.68 0 0.48	0.01 0 0.99	0.01 0 0.62

VI. Discussion

In figures 1, 2, and 3, we can see that all 3 models struggled to predict our target classes on the control data representing a traditional database, hovering just above 50%

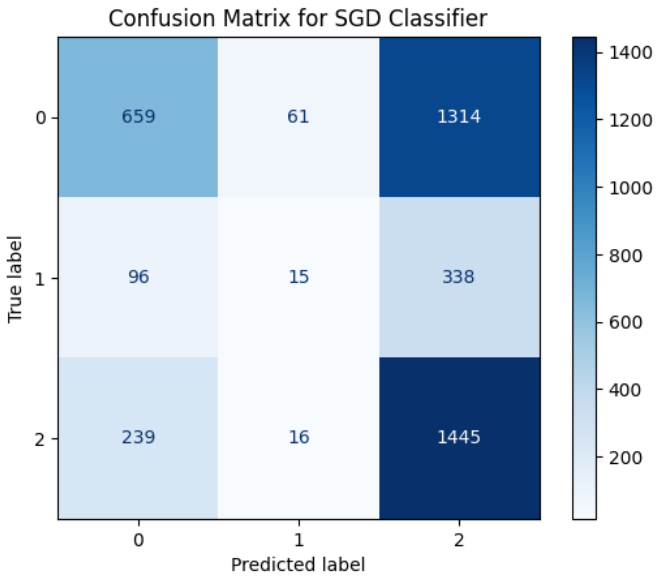

Fig. 1. SGDClassifier Confusion Matrix - Control

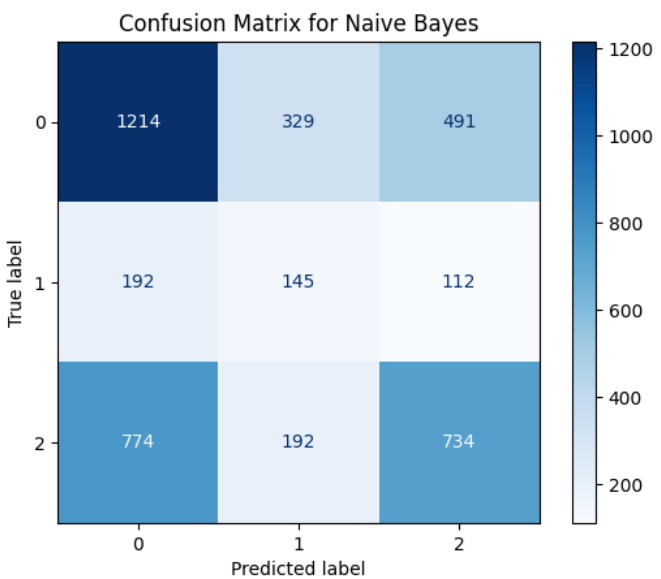

Fig. 2. MultinomialNB Confusion Matrix - Control

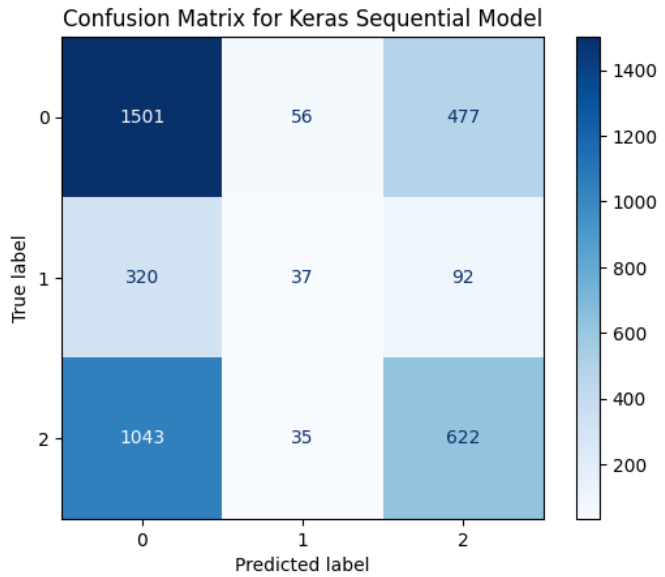

Fig. 3. Keras Sequential NN Confusion Matrix - Control

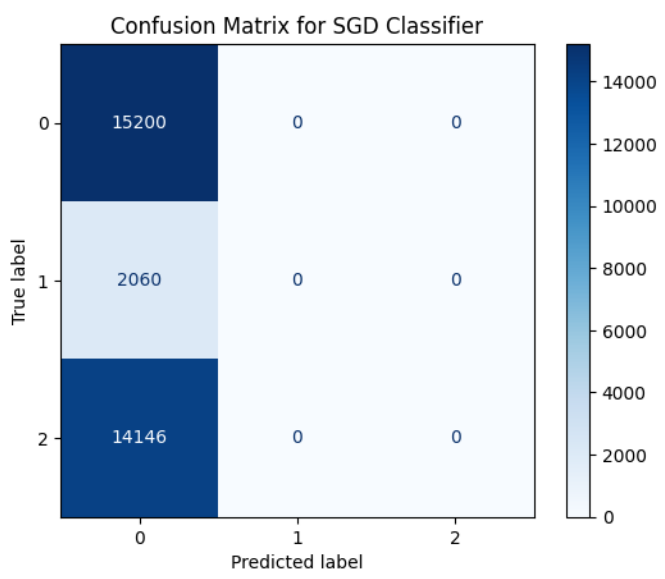

Fig. 4. SGDClassifier Confusion Matrix - Graph

accuracy. Also, the models particularly struggled with correctly classifying label 1, which represents the static case. Both the keras and Naive Bayes models (figures 2 and 3 respectively) disproportionately predicted a decrease and the the SGD classifier disproportionately predicted an increase. None of the models were able to do a particularly great job at predicting a specific class right any high rate of the time, instead generally defaulting to the class they predicted most often despite a reasonably balanced split between the increase and decrease classes as seen in table I.

In figures 4, 5 and 6 we see that all three models almost exclusively classified training samples as a decrease or increase, with the SGD classifier only ever classifying samples as a decrease; however, in table IV, we can see that this was still the classifier with the best accuracy. Once again, all 3 models rarely ever predicted that a record would be static.

When comparing model types between datasets, we actually see slightly worse performance by the models on the graph dataset, despite similar distributions in tables I and II. What we do see is all the models on the graph being extremely stuck on one of the 3 classes and predicting that class in all or the vast majority of cases.

There are several issues that may have impacted the experiment that will require further research. First, I think that

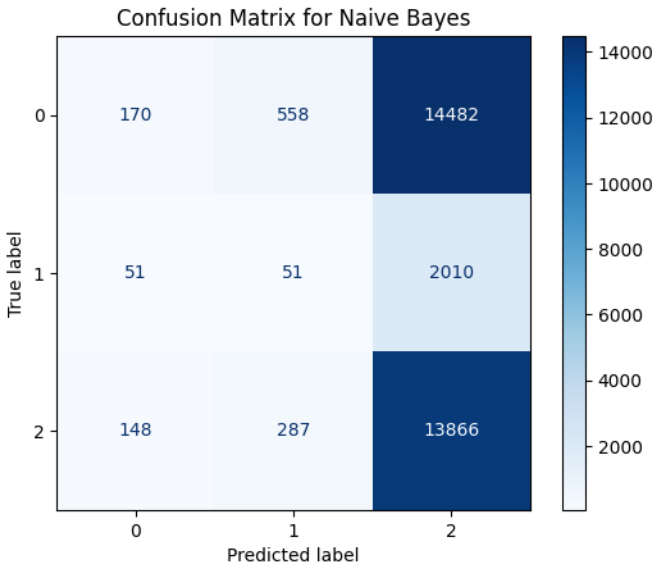

Fig. 5. MultinomialNB Confusion Matrix - Graph

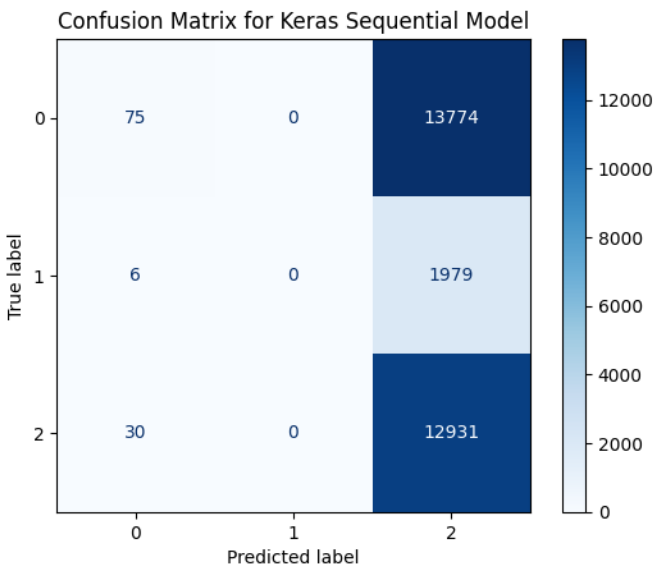

Fig. 6. Keras Sequential NN Confusion Matrix - Graph

trying a broader window in terms of what is considered the ”static” class may allow for a better balanced dataset; however, this is difficult because doing this would require constructing multiple graphs for each attempt, and building the graph has about 3 days of runtime. It is a similar issue with the chosen values for the most common generators, transporters, and treatment facilities as there may be large numbers of smaller entities, particularly with generators and transporters, that are having key effects that are not reflected in the graph. To solve this in a future experiment I believe I would focus on a specific region and include every entity within that region (although this would face another issue of missing things outside of but near whatever regional boundary was chosen). Finally, I think graph native ML methods may have a greater ability to handle the scale of this graph, but because the database is not a native graph database these are not available and would have to be built from scratch to work with my abstraction of a graph database which under the hood is still a relational DB with postgres.

VII. Conclusion

Based on this experiment I cannot conclude that organizing the data as a graph allows for greater predictive power as compared to a traditional database structure when predicting changes in waste amounts received by a given treatment facility; however, because of the potential issues mentioned in the discussion, there are still steps that would need to be taken in conjunction with this work in order to confirm that there is no advantage of using graph-based machine learning to predict these changes off of this data.

References

[1] “Rcrainfo download summary and data element dictionary,” *EPA*, [Online]. Available: https://echo.epa.gov/tools/data-downloads/rcrainfo-download-summary.

[2] “What is a graph database,” *neo4j*, [Online]. Available: https://neo4j.com/docs/getting-started/graph-database/.

[3] C. Russell, “Corporate strategies in the face of changing waste strategies,” *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/7465075.

[4] R. O. Richter, “Potential environmental impact of mercury emissions from portland cement kilns,” *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/1516358.

[5] L. Hui, “Design and application of remote i/o data acquisition equipment based on epa,” *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/5743226.

[6] P.-T. Chuang, “A two-dimensional expectation-perception analysis for rfid benefits in supply chain management,” *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/5668364.

[7] J. Bhatia, “Association rule mining by discretization of agricultural data using extended partitioning algorithm,” *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/9417954.

[8] M. Rakesh, “Enhanced medicare fraud detection using graph convolutional networks,” *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/10870744.

[9] J. Li, “Research on construction and application of supply chain trust knowledge graph under the background of intelligent manufacturing,” *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/11166242.

[10] P. Ghadekar, “A real-time gcn and ai agent framework for sustainable supply chain management under disaster scenarios,” *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/11306979.

[11] X. Ge, "Research and application of electronic data retrieval in material supply chain enhanced by large language models and knowledge graph," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/10936732.

[12] K. Gattupalli, "Cognitive supply chain innovations: Enabling self-sovereign identity and p2p lending platforms in vendor-managed systems," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/11132283.

[13] A. Chen, "Research on four-layer back propagation neural network for the computation of ship resistance," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/5245975.

[14] S. Xu, "Modeling of regional closed-loop international freight transportation network based on graph neural network," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/10894338.

[15] Q.-H. Ngo, "State estimation for power distribution system using graph neural networks," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/10220523.

[16] A. Chaudhary, "Forecasting retail sales demand by using autoregressive integrated moving average and graph neural network for supply chain optimization," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/11070740.

[17] k Akila, "Predictive analytics for supply chain optimization using ml models," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/11375956.

[18] V. S. Bobakov, "Classification of graph topologies by machine learning methods," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/10290449.

[19] Brandon-Rohrer, "Use postgres with python," [Online]. Available: https://www.brandonrohrer.com/postgres_intro.html.

[20] XD, "Using postgresql as a graph database: A simple approach for beginners," *Medium*, 2024.

[21] C. So, "I built an e(t)l-pipeline in python to transfer data from ms sql server to postgresql," *Medium*, 2021.

[22] H. Feng, "An approach to converting relational database to graph database: From mysql to neo4j," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/9719151.

[23] L. F. Ji, "Migrating data from relational database to graph database with tools," *IEEE*, [Online]. Available: https://ieeexplore.ieee.org/document/11329946.

[24] S. Sharma, "Building a graph database with vector embeddings: A python tutorial with neo4j and embeddings," *Medium*, 2024. [Online]. Available: https://medium.com/thedeephub/building-a-graph-database-with-vector-embeddings-a-python-tutorial-with-neo4j-and-embeddings-277ce608634d.

[25] "Incremental learning with scikit-learn," *GeeksForGeeks*, 2026. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/incremental-learning-with-scikit-learn/.

[26] A. Fanther, "Example: Simple python batch processing," *Medium*, 2025.

[27] K. Karunakaran, "Mastering keras' train_on_batch method: The ultimate guide for deep learning enthusiastsc," *Medium*, 2025.

[28] "Sgdclassifier," *scikit-learn*, [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html.

[29] "Multinomialnb," *scikit-learn*, [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html.

[30] fchollet, "The sequential model," *keras*, 2020.